

Lethesafe – Technical Whitepaper

Abstract

Lethesafe ist eine Sammlung lokal ausführbarer Werkzeuge zur Umsetzung von Time-Lock-Puzzles (TLP), die zeitverzögerte Geheimnisfreigaben ohne zentrale Infrastruktur ermöglicht. **Lethesafe verschlüsselt keine Geheimnisse im klassischen Sinn, sondern verzögert deterministisch die Existenz eines für den Zugriff notwendigen Schlüssels.** Optional kann die Verzögerung nutzerseitig zeitbasiert angegeben werden (z. B. Stunden oder Tage). Die hierfür notwendige Rundenzahl wird durch einen lokalen Kalibrierungslauf automatisch bestimmt; die kryptographischen Verfahren und Dateiformate bleiben unverändert. Das System kombiniert eine kryptographisch abgesicherte Hashkette mit passwortgeschützten Startwerten, einem Web-Frontend auf Flask-Basis sowie vollwertigen CLI-Werkzeugen. Alle kritischen Operationen laufen offline auf dem Gerät der Nutzer. Dieses Dokument beschreibt Designziele, Architektur, kryptographische Primitive, Sicherheitsannahmen und den aktuellen Entwicklungsstand.

1. Problemstellung & Zielbild

Zeitkapseln werden genutzt, um die Zielzeichenkette K erst nach einer definierten Menge an CPU-Arbeit entstehen zu lassen. Diese Zielzeichenkette kann anschließend als Masterpasswort für externe Verschlüsselungssysteme dienen. In Szenarien wie Notfall-Backups, digitaler Nachlassverwaltung oder erpresserischen Zugriffssituationen muss der Geheimnisträger sicherstellen, dass weder Angreifer noch er selbst vor Ablauf der Zeitbarriere Zugriff auf die verschlüsselten Informationen erlangen kann. Lethesafe adressiert dafür folgende Ziele:

- **Autarke Nutzung:** Keine Abhängigkeit von Servern, Blockchains oder Drittparteien.
- **Nachvollziehbarkeit:** Vollständig einsehbarer Python-Code in lethesafe_web und Entwicklung/.
- **Benutzerfreundlichkeit:** Browser-Oberfläche und CLI-Tools unterstützen wahlweise runden- oder zeitbasierte Eingaben zur Definition der Verzögerung.
- **Flexibilität:** Mehrere Rundenzahlen in einem Lauf, Cloning bestehender Kapseln und konfigurierbare Passwortpolitik.

2. Systemübersicht

Lethesafe besteht aus einer klar getrennten Komponentenlandschaft (Abb. 1):

1. **Core Library (lethesafe_web/lethesafe_core/)** – stellt create_capsules, clone_capsules und unlock_capsule bereit. Alle kryptographischen Primitive liegen in maker.py.
2. **Flask WebApp (lethesafe_web/app.py)** – exponiert API-Endpunkte (/api/create, /api/clone, /api/unlock, /api/run/cancel), verwaltet Downloads und orchestriert Hashläufe pro Anfrage.
3. **Frontend (lethesafe_web/static/lethesafe.js + CSS/Templates)** – führt Formvalidierung, Live-Fortschrittsanzeigen, Abbruchlogik und Result-Downloads im Browser aus.
4. **Abort-Manager (lethesafe_web/abort_manager.py)** – synchronisiert Cancel-Events über Run-Tokens.
5. **Progress-Tracker (lethesafe_web/progress_tracker.py)** – speichert temporär den Status laufender Hashketten und stellt ihn dem Frontend über /api/run/progress/<token> bereit.

6. **CLI-Suite (Entwicklung/lethesafe_maker.py, Entwicklung/lethesafe_unlocker.py, timelock_maker.py, unlocker.py)** – Offline-Generatoren und -Entschlüssler für Experten-Workflows, Grundlage für One-File Builds.
7. **Dokumentations- & Build-Hilfen** – info_readme/LIESMICH.txt beschreibt venv/PyInstaller-Pfade, Dokumentation/ liefert konzeptionelle Assets.

[User Action] → [Frontend] → [Flask API] → [Core Library] → [Result Cache / Files]



Abb.1

Während jeder Anfrage registriert app.py einen Run-Token. abort_manager liefert should_abort, während der Progress-Tracker kontinuierlich completed_rounds/elapsed_time speichert, die das Frontend per /api/run/progress/<token> abrufen kann. Alle Komponenten laufen lokal; Netzwerkzugriffe sind ausschließlich für optionale Updates notwendig.

3. Kryptographische Grundlagen

3.1 Zufallsquellen & Entropie

- create_capsules nutzt _derive_entropy (SHA-512 über Label, os.urandom und optionale Nutzerentropie) zur Initialisierung des Startwerts S sowie zur Ableitung der Zielzeichenkette K. Die Nutzerentropie erhöht dabei ausschließlich die Zufälligkeit des Initialzustands und hat keinen Einfluss auf die Dauer oder Struktur der Zeitverzögerung.
- Optional sammelt das Web-Frontend Mausbewegungen (create.html + JS) als zusätzliche Entropiequelle.

3.2 Hashketten & Ziele

- _hash_chain_targets berechnet deterministisch eine SHA-256-Kette über S, bis die maximale Rundenzahl erreicht ist.
- Die Funktion akzeptiert eine should_abort-Callback, sodass laufende Hashketten sicher beendet werden können.

3.3 Zielzeichenketten-Ableitung

Für jede Runde r gilt:

$$H_r = \text{SHA256}^r(S)$$

$$\text{Puzzle}_r = K \oplus H^r$$

Die Zielzeichenkette K ist an das jeweilige Hashziel H_r gebunden. Nur durch das vollständige sequenzielle Durchlaufen der Hashkette kann H_r rekonstruiert und daraus mittels XOR-Operation die Zielzeichenkette K gewonnen werden.

Die XOR-Operation dient ausschließlich der Bindung von K an das Hashziel und stellt keine Verschlüsselung semantischer Daten dar. Lethesafe verarbeitet zu keinem Zeitpunkt vertrauliche Informationen, sondern verzögert ausschließlich die Entstehung der Zielzeichenkette K.

3.4 Passwortgeschützter Startwert

_protect_start schützt S mit folgenden Bausteinen:

Schritt	Verfahren	Zweck
Key-Derivation	PBKDF2-HMAC-SHA256 mit 400.000 Iterationen	Resistenz gegen Brute-Force
Keystream	BLAKE2b (person=LethSenc)	One-Time-Pad für S
Authentizität	HMAC-SHA256 mit BLAKE2b-abgeleitetem Schlüssel (person=LethSmac)	Schutz vor Manipulation

Das resultierende Objekt `start_value_protected` enthält ciphertext, salt, nonce, iterations und mac (Base64). `_decrypt_start` verifiziert MACs via `hmac.compare_digest` und liefert S nur bei korrektem Passwort.

Die Iterationszahl ist bewusst als Parameter Teil des Puzzle-Formats, um zukünftige Anpassungen an Hardwareentwicklungen zu ermöglichen, ohne bestehende Kapseln zu invalidieren.

Die Verwendung eines Start-Passworts ist optional und dient ausschließlich dem Schutz vor unbefugtem Starten des Hashprozesses; sie hat keinen Einfluss auf die Dauer oder Sicherheit der Zeitverzögerung selbst.

3.5 Puzzle-Dateiformat

Jede erzeugte JSON-Datei folgt dem in `maker.py` definierten Schema:

```
{
  "format": "lethesafe-puzzle",
  "version": 2,
  "mode": "new" | "clone",
  "hash_function": "sha256",
  "rounds": 20000,
  "puzzle": "<Base64>",
  "start_value_protected": { ... },
  "secret_checksum": "<SHA256(K)>",
  "start_value_protection": "password_pbkdf2_blake2b_hmac",
  "start_value_iterations": 400000
}
```

Die Versionierung erlaubt spätere Erweiterungen (z. B. alternative Hashfunktionen oder KDF-Parameter). Die `secret_checksum` von K dient zur Plausibilitätsprüfung bei Cloning-Läufen.

Zusätzlich können Puzzle-Dateien optionale Metadaten enthalten, die dokumentieren, **wie** die Rundenzahl zustande gekommen ist (z. B. direkte Angabe durch den Nutzer oder Ableitung aus einer zeitbasierten Eingabe). Diese Metadaten dienen ausschließlich der Transparenz und Nachvollziehbarkeit. Sie haben **keinen Einfluss auf die Entschlüsselung, die kryptographische Sicherheit oder die Gültigkeit des Puzzle-Formats**.

4. Workflow-Design

4.1 Erstellung (/api/create)

1. Frontend validiert Eingaben, sammelt Entropie und initialisiert den Lauf; Zeitabschätzungen erfolgen ausschließlich während des laufenden Hashprozesses.

2. Nach Formular-Submit erzeugt der Browser ein Lauf-Token (crypto.randomUUID) und sendet dieses in X-Run-Token-Header.
3. Flask registriert das Token beim abort_manager, ruft create_capsules, cached Resultate (_RESULT_CACHE) und liefert Download-Tokens für [K] zur Verwendung als externes Masterpasswort, Passwortdatei und Puzzle-Dateien.
4. Browser bietet Dateien via /download/<token> als Blob an; eine passwortfreie zeitkapsel_secret.txt dient dem Offline-Export.

Zeitbasierter Modus (optional):

Alternativ zur direkten Angabe der Rundenzahl kann der Nutzer eine gewünschte Zeitdauer angeben. Das System führt vor dem eigentlichen Hashlauf einen kurzen lokalen Kalibrierungslauf durch, um die reale Hashrate des Geräts zu bestimmen und daraus die erforderliche Rundenzahl abzuleiten. Der eigentliche Hashprozess startet erst nach expliziter Bestätigung dieser Parameter.

4.2 Cloning (/api/clone)

- Eingeladene Puzzle-Datei wird geparkt (_load_puzzle).
- unlock_capsule-Logik rekonstruiert S und K (erneuter Hashlauf über die Originalrunden).
- Nutzer entscheiden, ob das ursprüngliche Passwort weiterverwendet oder via _protect_start ersetzt wird.
- Neue Runden werden berechnet und als eigenständige JSON-Dateien exportiert.

4.3 Entschlüsselung (/api/unlock)

- Die API akzeptiert Puzzle-Datei + Passwort, validiert Größe (MAX_CONTENT_LENGTH = 32 * 1024 * 1024 in app.py) und ruft unlock_capsule.
- Das Ergebnis wird ausschließlich Base64-kodiert ausgeliefert. Optionaler Download liefert zeitkapsel_secret.txt mit [K] und Timestamp.

4.4 Laufabbruch & UX

- Frontend-Seiten nutzen AbortController und navigator.sendBeacon (lethesafe.js) um /api/run/cancel zu triggern.
- Progressbars fragen /api/run/progress/<token> ab und berechnen die ETA ausschließlich aus den tatsächlich abgeschlossenen Runden (keine persistente Hashraten-Schätzung).
- Navigation wird solange blockiert, bis der Lauf abgeschlossen oder abgebrochen ist (BeforeUnload-Guard).

5. Sicherheitsbetrachtung

Angriff/Fehlerfall	Gegenmaßnahme
Gestohlene Puzzle-Datei	Hashlauf zwingend nötig; Startwert ist ohne Passwort nicht rekonstruierbar.
Manipulierte JSON-Struktur	maker.py validiert Felder, _load_puzzle prüft Format und Base64-Dekodierbarkeit.
Brute-Force auf Passwort	PBKDF2 mit 400k Iterationen und BLAKE2b-MAC erhöht Aufwand; unbegrenzte Eingabeversuche sollen legitime Anwender nicht ausbremsen, die Sicherheit der Zeitverzögerung ist unabhängig von der Wahl oder Existenz eines Start-Passworts.

Fehlhafter Upload-Requests	Upload-Größenlimits und Validierung von request.files verhindern die Verarbeitung unvollständiger oder missbräuchlicher HTTP-Anfragen.
Nutzerinitiiertes Laufabbruch	Run-Tokens + should_abort Callback erlauben einen jederzeitigen, kontrollierten Abbruch laufender Hashprozesse ohne Seiteneffekte oder Speicherlecks.
Wiederverwendung identischer Dateinamen	CLI (lethesafe_maker.py) erzeugt eindeutige Namen (build_filename). Web-Ausgaben sind tokenisiert und überschreiben nichts.
Result Leakage via Cache	_RESULT_CACHE lebt nur im Prozess und wird durch Browser-Downloads geleert (kein persistent Storage).

Threat Model Annahmen - Der Client kann die Laufzeit nicht beschleunigen, da der verwendete SHA256-Hashlauf inhärent sequenziell ist. - Sobald [K] einmal offengelegt wurde, gilt es als temporär existent und muss nach der beabsichtigten Verwendung vollständig vernichtet werden. Jede weitere Existenz – sei es als Datei, Kopie oder gespeicherte Zeichenfolge – hebt den zeitlichen Schutzmechanismus vollständig auf. - Hardware-Seitenkanäle (z. B. CPU Voltage-Glitches) liegen außerhalb des aktuellen Scopes.

Die zeitbasierte Parametrisierung stellt keine zusätzliche Angriffsfläche dar, da sie ausschließlich lokal erfolgt und ausschließlich zur Berechnung der Rundenzahl dient.

6. CLI-Tools & Offline-Modus

6.1 lethesafe_maker.py

- Vollständig textbasiert, erlaubt „Neu“ und „Clone“-Modus, zeigt [K] an und schreibt wahlweise Secrets auf Disk.
- Enthält Komfortfunktionen wie prompt_rounds, sanitize_basename, compute_hashes_for_rounds und userfreundliche Emojis.
- Unterstützt optional eine zeitbasierte Verzögerungseingabe (z. B. 6h, 3d) mit lokaler Kalibrierung der Hashrate.
- Die berechnete Rundenzahl wird vor Start angezeigt und als regulärer Parameter in der Zeitkapsel gespeichert.
- Exportiert Metadaten (mode, source_file, secret_checksum), damit Web- und CLI-Dateien kompatibel bleiben.
- Zeigt eine Live-ETA basierend auf der tatsächlich erreichten Rundenzahl und erlaubt unbegrenzt viele Passwortversuche (Fehleingaben führen lediglich zu einer erneuten Abfrage).

6.2 lethesafe_unlocker.py

- Fragt das Zeitkapsel-Passwort ab, validiert Startwerte und speichert [K] optional als key_<rounds>.txt inklusive UTC-Timestamp.
- Verwendet dieselbe Live-ETA-Ausgabe wie der Maker und begrenzt Passwortversuche nicht.

7. Betriebs- & Deployment-Aspekte

- **Lokalbetrieb:** FLASK_APP=app.py flask run im Verzeichnis lethesafe_web reicht aus; keine DB oder externen Dienste nötig.

- **Result-Downloads:** _RESULT_CACHE speichert (Dateiname, Bytes) im Speicher. Downloads erfolgen über /download/<token> mit send_file.
- **Ressourcenbegrenzung:** MAX_CONTENT_LENGTH verhindert DoS über riesige Uploads, Hashketten laufen rein CPU-bound.
- **Internationalisierung:** Default-Sprache ist Deutsch; Strings sind in Templates/JS hardcodiert. Mehrsprachigkeit ist als künftige Erweiterung angedacht.

8. Performance & Nutzererlebnis

- Browserseitige ETA-Anzeigen basieren ausschließlich auf Live-Daten aus dem Progress-Tracker (completed_rounds, elapsed_time). Historische Hashraten oder localStorage-Schätzungen werden nicht mehr verwendet.
- Zeitbasierte Verzögerungen verwenden eine kurze Vorabmessung der Hashrate; Fortschrittsanzeigen und ETA während des eigentlichen Hashlaufs basieren weiterhin ausschließlich auf real abgeschlossenen Runden.
- Fortschrittsanzeigen zeigen getrennte Phasen (z. B. clone.html unterscheidet Original-Hashlauf und neue Kapseln) und aktualisieren sich in Echtzeit über /api/run/progress/<token>.
- Passwort-Eingaben sind bewusst nicht limitiert; falsche Eingaben resultieren lediglich in einer erneuten Abfrage, sowohl im Web-Frontend als auch in den CLI-Tools.
- Mausektronie-Aufzeichnung limitiert sich auf 200 Punkte und wird gedrosselt (alle 40 ms), um UI-Responsiveness zu erhalten.

9. Roadmap & Erweiterungsmöglichkeiten

Basierend auf info_readme/feature-ideen.txt und offenen Punkten im Code:

1. **CLI-Automatisierung:** Parameterbasierte Nutzung ohne Prompting (Batch-Skripte, CI).
2. **Erweiterte Validierung:** Zusätzliche Format-/Integritätschecks vor langen Hashläufen (z. B. JSON-Schema oder Signaturen).
3. **Datei-Workflow-Härtung:** Optionale Vorabdefinition des Secret-Speicherorts (Schutz gegen Ausfälle am Laufende).
4. **UPX/Packaging-Optimierung:** Kleinere Binaries für Ressourcenkontexte.
5. **Kryptographische Agilität:** Austauschbare Hashfunktionen oder zukünftige VDF-Module, sobald erforderlich.
6. **Mehrsprachige UI & Dokumentation:** Internationalisierung der Templates und JS-Strings.
7. **Feingranulare Telemetrie:** Optionale Hooks zur lokalen Laufzeitmessung, basierend auf dem Progress-Tracker.

10. Fazit

Lethesafe verbindet bewährte Kryptographie (SHA256, PBKDF2, BLAKE2b, HMAC) mit praxisnahen Bedienkonzepten. Sowohl Web- als auch CLI-Tools garantieren, dass Zeitkapseln komplett lokal entstehen, geklont und entschlüsselt werden – ohne Vertrauen in den Hersteller. Das Projekt ist bereit für Audits, Community-Beiträge und produktive Nutzung auf Desktops oder spezialisierten Devices. Nächste Schritte bestehen darin, Automatisierungsschnittstellen, zusätzliche Validierung und optionale Mehrsprachigkeit zu liefern, um Lethesafe als Referenzimplementierung für Time Lock-Puzzles zu etablieren.

Dieses Dokument beschreibt die technische Architektur, die kryptographischen Grundlagen und die Sicherheitsannahmen von Lethesafe in ihrem stabilen konzeptionellen Zustand. Änderungen an Benutzeroberfläche, Implementierungsdetails oder Performance-Optimierungen haben keine Auswirkungen auf die hier beschriebenen Prinzipien

Copyright © 2026 Ronald Stegmiller